

MASTER COMMAND PROMPT: TryNex Lifestyle (trynexshop.com) — Complete Platform Overhaul

CRITICAL INSTRUCTION: Do NOT skip any section. Do NOT partially implement. Execute ALL fixes below in a single pass. Every section is mandatory. This is a print-on-demand e-commerce platform built with Next.js/React. The site has severe issues in the Design Studio, Frontend Responsiveness, Admin Panel, and Backend. Fix everything to production-grade, premium quality — matching or exceeding Printful/Printify standards.

SECTION 1: DESIGN STUDIO COMPLETE REBUILD (/customize page)

1.1 — Canvas & Interaction System (HIGHEST PRIORITY)

Problem: Users cannot touch, click, hold, drag, move, or resize uploaded designs. The interaction system is broken on both mobile and desktop.

Fix — Implement a Unified Pointer Event System:

Plain Text

- Replace ALL mouse-only event listeners (onClick, onMouseDown, onMouseMove, on) with PointerEvents API (onPointerDown, onPointerMove, onPointerUp, onPointerC
- This single API handles mouse, touch, and stylus simultaneously
- Add CSS: `touch-action: none` on the canvas/design area container to prevent browser default touch behaviors (scrolling, zooming) from interfering
- Implement the following interaction states:
 1. IDLE: No design selected
 2. SELECTED: Design has selection handles (resize corners + rotation handle)
 3. DRAGGING: User is holding and moving the design
 4. RESIZING: User is dragging a corner handle
 5. ROTATING: User is dragging the rotation handle
- On pointerdown inside a design element → set state to SELECTED, show handles
- On pointerdown + pointermove on the design body → set state to DRAGGING, update position with (currentPointerX - offsetX, currentPointerY - offsetY)
- On pointerdown + pointermove on a corner handle → set state to RESIZING, calculate new width/height maintaining aspect ratio
- On pointerup → finalize position, save coordinates
- On pointerdown OUTSIDE the design/print area → DESELECT the design, hide handles, finalize placement

Implementation Details:

JavaScript

```
// Example structure for the design canvas component
const DesignCanvas = () => {
  const [designs, setDesigns] = useState([]);
  const [activeDesign, setActiveDesign] = useState(null);
  const [isDragging, setIsDragging] = useState(false);
  const [dragOffset, setDragOffset] = useState({ x: 0, y: 0 });
  const canvasRef = useRef(null);

  const handlePointerDown = (e, designId) => {
    e.preventDefault();
    e.stopPropagation();
    e.target.setPointerCapture(e.pointerId); // Critical for smooth dragging
    setActiveDesign(designId);
    setIsDragging(true);
    const rect = e.target.getBoundingClientRect();
    setDragOffset({ x: e.clientX - rect.left, y: e.clientY - rect.top });
  };

  const handlePointerMove = (e) => {
    if (!isDragging || !activeDesign) return;
    const canvasRect = canvasRef.current.getBoundingClientRect();
    const newX = e.clientX - canvasRect.left - dragOffset.x;
    const newY = e.clientY - canvasRect.top - dragOffset.y;
    // Clamp within print area bounds
    updateDesignPosition(activeDesign, clampToPrintArea(newX, newY));
  };

  const handlePointerUp = (e) => {
    setIsDragging(false);
  };

  const handleCanvasClick = (e) => {
    // If click is on canvas background (not on a design), deselect
    if (e.target === canvasRef.current) {
      setActiveDesign(null);
    }
  };

  return (
    <div
      ref={canvasRef}
      onPointerMove={handlePointerMove}
      onPointerUp={handlePointerUp}
      onClick={handleCanvasClick}
    />
  );
};
```

```

    style={{ touchAction: 'none', position: 'relative' }}
  >
    {designs.map(design => (
      <DesignElement
        key={design.id}
        design={design}
        isActive={activeDesign === design.id}
        onPointerDown={(e) => handlePointerDown(e, design.id)}
      />
    ))}
  </div>
);
};

```

1.2 — Print Area Correction (CRITICAL)

Problem: Every product shows WRONG print area. The print zone borders don't match actual product printable dimensions. This is the biggest mistake on the site.

Fix — Define Accurate Print Areas for Each Product:

Plain Text

Create a configuration file: /src/config/printAreas.js (or .ts)

Each product MUST have:

- productId: unique identifier
- name: product name
- views: array of available views (front, back, left_sleeve, right_sleeve, neck)
- For each view:
 - mockupImageWidth: the width of the mockup image in pixels
 - mockupImageHeight: the height of the mockup image in pixels
 - printArea: { x, y, width, height } in PERCENTAGE of the mockup image
 - physicalPrintSize: { widthInches, heightInches } – the actual real-world print size
 - aspectRatio: calculated from physicalPrintSize

CORRECT PRINT AREA DIMENSIONS (industry standard):

T-SHIRT (Front):

- Physical print area: 12" wide × 16" tall (standard DTG full front)
- Safe area: 11" wide × 14" tall
- The print zone on the mockup should be positioned at approximately:
 - x: 28% from left, y: 22% from top, width: 44%, height: 52%
 (These percentages are relative to the full mockup image)

T-SHIRT (Back):

- Physical print area: 12" wide × 16" tall
- Same positioning as front

T-SHIRT (Left/Right Sleeve):

- Physical print area: 3.5" wide × 4" tall
- Position: centered on sleeve area of mockup

HOODIE (Front):

- Physical print area: 12" wide × 12" tall (shorter due to kangaroo pocket)
- Position: x: 27%, y: 20%, width: 46%, height: 40%

HOODIE (Back):

- Physical print area: 12" wide × 14" tall
- Position: x: 27%, y: 18%, width: 46%, height: 48%

MUG (Wrap):

- Physical print area: 8.5" wide × 3.5" tall (standard 11oz mug)
- The mug mockup should show a flattened/curved print zone

CAP (Front Panel):

- Physical print area: 4" wide × 2.5" tall
- Position: centered on front panel of cap mockup

Implementation:

JavaScript

```
// /src/config/printAreas.js
export const PRODUCT_PRINT_AREAS = {
  'unisex-tshirt': {
    name: 'Unisex T-Shirt',
    views: {
      front: {
        printArea: { x: 28, y: 22, width: 44, height: 52 }, // percentages
        physicalSize: { width: 12, height: 16, unit: 'inches' },
        aspectRatio: 12 / 16, // 0.75
      },
      back: {
        printArea: { x: 28, y: 22, width: 44, height: 52 },
        physicalSize: { width: 12, height: 16, unit: 'inches' },
        aspectRatio: 12 / 16,
      },
      left_sleeve: {
        printArea: { x: 10, y: 30, width: 18, height: 20 },
        physicalSize: { width: 3.5, height: 4, unit: 'inches' },
        aspectRatio: 3.5 / 4,
      },
      right_sleeve: {
        printArea: { x: 72, y: 30, width: 18, height: 20 },
        physicalSize: { width: 3.5, height: 4, unit: 'inches' },
      }
    }
  }
}
```

```

        aspectRatio: 3.5 / 4,
    },
},
'unisex-hoodie': {
    name: 'Unisex Hoodie',
    views: {
        front: {
            printArea: { x: 27, y: 20, width: 46, height: 40 },
            physicalSize: { width: 12, height: 12, unit: 'inches' },
            aspectRatio: 1,
        },
        back: {
            printArea: { x: 27, y: 18, width: 46, height: 48 },
            physicalSize: { width: 12, height: 14, unit: 'inches' },
            aspectRatio: 12 / 14,
        },
    },
},
'coffee-mug': {
    name: 'Coffee Mug',
    views: {
        wrap: {
            printArea: { x: 15, y: 25, width: 70, height: 50 },
            physicalSize: { width: 8.5, height: 3.5, unit: 'inches' },
            aspectRatio: 8.5 / 3.5,
        },
    },
},
'structured-cap': {
    name: 'Structured Cap',
    views: {
        front: {
            printArea: { x: 30, y: 20, width: 40, height: 35 },
            physicalSize: { width: 4, height: 2.5, unit: 'inches' },
            aspectRatio: 4 / 2.5,
        },
    },
},
};

```

1.3 — Auto-Fit Algorithm for Uploaded Images

Problem: Uploaded pictures don't auto-fit correctly within the print area. They overflow, are too small, or don't maintain aspect ratio.

Fix:

JavaScript

```
// When a user uploads an image, calculate the optimal fit:
function autoFitDesign(imageWidth, imageHeight, printAreaWidth, printAreaHeight) {
  const imageAspect = imageWidth / imageHeight;
  const areaAspect = printAreaWidth / printAreaHeight;

  let fitWidth, fitHeight;

  if (imageAspect > areaAspect) {
    // Image is wider than print area – fit to width
    fitWidth = printAreaWidth * 0.9; // 90% of print area width (with margin)
    fitHeight = fitWidth / imageAspect;
  } else {
    // Image is taller than print area – fit to height
    fitHeight = printAreaHeight * 0.9; // 90% of print area height (with margin)
    fitWidth = fitHeight * imageAspect;
  }

  // Center the design within the print area
  const x = (printAreaWidth - fitWidth) / 2;
  const y = (printAreaHeight - fitHeight) / 2;

  return { x, y, width: fitWidth, height: fitHeight };
}
```

1.4 — Color Change Sizing Bug Fix

Problem: When users change the product color, the mockup image container changes size (becomes larger or smaller), causing layout shifts.

Fix:

CSS

```
/* Apply to the mockup container */
.mockup-container {
  position: relative;
  width: 100%;
  max-width: 500px; /* Fixed maximum width */
  aspect-ratio: 1 / 1; /* Or the exact ratio of your mockup images */
  margin: 0 auto;
  overflow: hidden;
}

.mockup-image {
  position: absolute;
  top: 0;
```

```

left: 0;
width: 100%;
height: 100%;
object-fit: contain; /* CRITICAL – prevents stretching */
}

/* The print area overlay must use percentage-based positioning */
.print-area-overlay {
position: absolute;
/* Use percentages from the config, e.g.: */
top: 22%;
left: 28%;
width: 44%;
height: 52%;
border: 2px dashed rgba(0, 150, 255, 0.6);
pointer-events: none; /* Don't block interactions with designs inside */
}

```

JavaScript fix:

JavaScript

```

// When color changes, ONLY swap the image source – do NOT re-render the container
const handleColorChange = (newColor) => {
  // Just update the mockup image URL
  setMockupImage(getColorMockupUrl(product.id, currentView, newColor));
  // Do NOT change container dimensions
  // Do NOT recalculate print area
  // The print area stays exactly the same regardless of color
};

```

1.5 — Design Overlay Realism (Product Skin Integration)

Problem: Uploaded designs look like they're floating ON TOP of the product instead of being printed on it.

Fix — Implement Blend Mode Technique:

CSS

```

/* The uploaded design should blend with the product texture */
.design-layer {
position: absolute;
/* Position within print area */
mix-blend-mode: multiply; /* Makes the design look printed on fabric */
opacity: 0.92; /* Slight transparency for realism */
}

```

```

/* For light-colored products, use multiply. For dark products, use screen */
.design-layer--dark-product {
  mix-blend-mode: screen;
  opacity: 0.88;
}

```

Advanced technique (optional but recommended):

JavaScript

```

// Use a product texture overlay on top of the design
// Layer order (bottom to top):
// 1. Product mockup base (colored)
// 2. Uploaded design (with mix-blend-mode)
// 3. Product shadow/texture overlay (semi-transparent, multiply blend)

<div className="mockup-container">
  <img src={mockupBase} className="mockup-image" /> {/* Layer 1 */}
  <div className="print-area">
    <img src={uploadedDesign} className="design-layer" /> {/* Layer 2 */}
  </div>
  <img src={mockupOverlay} className="texture-overlay" /> {/* Layer 3 - shadows
</div>

```

SECTION 2: FRONTEND RESPONSIVENESS — ALL PAGES

2.1 — Global CSS Reset & Responsive Foundation

Problem: The entire website has responsiveness issues. Horizontal overflow, broken layouts on mobile, scroll issues on desktop.

Fix — Add/Update Global Styles:

CSS

```

/* globals.css or layout.css — ADD THESE AT THE TOP */

*, *::before, *::after {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

html {

```



```

overflow-x: hidden; /* Prevent horizontal scroll on the HTML element */
scroll-behavior: smooth;
}

body {
  overflow-x: hidden; /* Prevent horizontal scroll */
  -webkit-overflow-scrolling: touch; /* Smooth momentum scrolling on iOS */
  min-height: 100vh;
}

/* REMOVE any overflow: hidden on main content wrappers that blocks vertical sc
/* Search for and remove these problematic patterns: */
/* .main-wrapper { overflow: hidden; } ← REMOVE THIS */
/* .page-container { overflow: hidden; } ← REMOVE THIS */
/* Only keep overflow: hidden on specific small containers where needed */

img, video {
  max-width: 100%;
  height: auto;
  display: block;
}

/* Responsive container */
.container {
  width: 100%;
  max-width: 1280px;
  margin: 0 auto;
  padding: 0 16px;
}

@media (min-width: 768px) {
  .container { padding: 0 24px; }
}

@media (min-width: 1024px) {
  .container { padding: 0 32px; }
}

```

2.2 — Scroll Fix (Desktop Mouse Wheel + Mobile Touch)

Problem: Mouse scroll doesn't work properly on desktop. Touch scroll broken on mobile in certain areas.

Fix:

Plain Text

1. Search the ENTIRE codebase for:
 - overflow: hidden (on any element larger than a small card/modal)
 - position: fixed (on non-modal elements that might block scroll)
 - height: 100vh (on content wrappers – this prevents scrolling)
 - Any custom scroll libraries (e.g., locomotive-scroll, smooth-scrollbar)
2. For each found instance:
 - If it's on a page wrapper/main content → REMOVE IT
 - If it's on a modal/dropdown → KEEP IT but add scroll inside
 - If it's a custom scroll library → REMOVE IT entirely, use native scroll
3. Ensure the design studio page specifically allows:
 - Vertical page scroll OUTSIDE the canvas area
 - NO scroll INSIDE the canvas area (to prevent accidental scrolling while de
 - Add: canvas area → touch-action: none; overflow: hidden;
 - Add: page body → overflow-y: auto; (normal scroll)

2.3 — Page-by-Page Responsive Fixes

Homepage:

Plain Text

- Hero section: Use responsive font sizes (clamp(1.5rem, 4vw, 3.5rem))
- Product grid: Use CSS Grid with auto-fill: grid-template-columns: repeat(auto
- Testimonials: Horizontal scroll on mobile with snap: scroll-snap-type: x mand
- All sections: padding responsive with clamp()

Shop/Products Page:

Plain Text

- Filter sidebar: Collapsible on mobile (slide-in drawer pattern)
- Product cards: 2 columns on mobile, 3 on tablet, 4 on desktop
- Ensure product images all have consistent aspect-ratio: 1/1 containers

Product Detail Page:

Plain Text

- Image gallery: Full-width on mobile, 50% on desktop
- Sticky "Add to Cart" bar on mobile (fixed bottom)
- Size selector: Large touch targets (min 44px × 44px)

Cart & Checkout:

Plain Text

- Single column on mobile
- Clear quantity controls with large +/- buttons
- Order summary sticky on desktop sidebar

2.4 — Image Optimization

Fix:

JavaScript

```
// Replace ALL <img> tags with Next.js <Image /> component
import Image from 'next/image';

// Before (broken):


// After (correct):
<Image
  src="/products/tshirt.png"
  alt="Unisex T-Shirt"
  width={500}
  height={500}
  sizes="(max-width: 768px) 100vw, (max-width: 1200px) 50vw, 33vw"
  style={{ objectFit: 'contain' }}
  priority={isAboveFold}
/>
```

SECTION 3: ADMIN PANEL FIXES

3.1 — Database Connection Issue

Problem: Admin dashboard shows "Database: Not Connected" in some places but "Connected" in others.

Root Cause: Likely a serverless cold-start issue or environment variable misconfiguration.

Fix:

Plain Text

1. Check your database connection code. If using Prisma:

// prisma/client.ts or lib/prisma.ts

```
import { PrismaClient } from '@prisma/client';

const globalForPrisma = globalThis as unknown as { prisma: PrismaClient };

export const prisma = globalForPrisma.prisma || new PrismaClient({
  log: ['error', 'warn'],
});

if (process.env.NODE_ENV !== 'production') globalForPrisma.prisma = prisma;

// This prevents creating new connections on every serverless invocation
```

2. If using a direct PostgreSQL/MySQL connection:

- Implement connection pooling
- Add retry logic with exponential backoff
- Use a connection health check before queries

3. Fix the dashboard status indicator:

```
// Create an API endpoint: /api/admin/health
export async function GET() {
  try {
    await prisma.$queryRaw`SELECT 1`; // Simple health check
    return Response.json({ database: 'connected', timestamp: Date.now() });
  } catch (error) {
    return Response.json({ database: 'disconnected', error: error.message },
  }
}

// In the admin dashboard, call this endpoint on mount:
useEffect(() => {
  fetch('/api/admin/health')
    .then(res => res.json())
    .then(data => setDbStatus(data.database))
    .catch(() => setDbStatus('disconnected'));
}, []);
```

4. Environment Variables – Verify in Replit Secrets:

- DATABASE_URL must be set correctly
- If using Neon/Supabase/PlanetScale, ensure the connection string includes:
?sslmode=require&connection_limit=5&pool_timeout=20

3.2 — Admin Panel Responsiveness

Fix:

CSS

```
/* Admin layout should use a collapsible sidebar pattern */
.admin-layout {
  display: grid;
  grid-template-columns: 1fr;
  min-height: 100vh;
}

@media (min-width: 1024px) {
  .admin-layout {
    grid-template-columns: 260px 1fr;
  }
}

/* Admin sidebar – hidden on mobile, slide-in on toggle */
.admin-sidebar {
  position: fixed;
  left: -260px;
  top: 0;
  width: 260px;
  height: 100vh;
  transition: left 0.3s ease;
  z-index: 50;
}

.admin-sidebar.open {
  left: 0;
}

@media (min-width: 1024px) {
  .admin-sidebar {
    position: static;
    left: 0;
  }
}

/* Admin tables – horizontal scroll on mobile */
.admin-table-wrapper {
  overflow-x: auto;
  -webkit-overflow-scrolling: touch;
}

/* Admin cards/stats – stack on mobile */
.admin-stats-grid {
  display: grid;
  grid-template-columns: 1fr;
  gap: 16px;
}
```

```
@media (min-width: 640px) {  
  .admin-stats-grid { grid-template-columns: repeat(2, 1fr); }  
}  
  
@media (min-width: 1024px) {  
  .admin-stats-grid { grid-template-columns: repeat(4, 1fr); }  
}
```

SECTION 4: PREMIUM UI/UX POLISH

4.1 — Design System Consistency

Plain Text

Apply these across the ENTIRE site:

Typography:

- Headings: font-family: 'Inter', sans-serif (or your brand font)
- Body: font-family: 'Inter', sans-serif
- Font sizes: Use a modular scale (1rem, 1.125rem, 1.25rem, 1.5rem, 2rem, 2.5rem)

Spacing:

- Use 4px base unit: 4, 8, 12, 16, 24, 32, 48, 64, 96px
- Consistent padding on all sections: py-16 (mobile), py-24 (desktop)

Colors:

- Ensure sufficient contrast ratios (WCAG AA: 4.5:1 for text)
- Consistent use of brand colors
- Hover states on all interactive elements

Shadows:

- Cards: box-shadow: 0 1px 3px rgba(0,0,0,0.1), 0 1px 2px rgba(0,0,0,0.06)
- Elevated: box-shadow: 0 10px 15px rgba(0,0,0,0.1), 0 4px 6px rgba(0,0,0,0.05)

Border Radius:

- Buttons: 8px
- Cards: 12px
- Inputs: 8px
- Consistent everywhere

Transitions:

- All interactive elements: transition: all 0.2s ease

- Buttons: `scale(0.98)` on active
- Cards: `translateY(-2px)` on hover

4.2 — Loading States & Error Handling

Plain Text

- Add skeleton loaders for all data-fetching components
- Add proper error boundaries
- Add toast notifications for user actions (design saved, added to cart, etc.)
- Add loading spinners for image uploads in the design studio
- Show upload progress percentage

4.3 — Performance Optimization

Plain Text

- Lazy load all below-fold images
- Code-split the design studio (it's heavy – load it dynamically)
- Minimize bundle size – check for unused dependencies
- Add proper meta tags and SEO
- Implement proper caching headers

SECTION 5: TESTING CHECKLIST (VERIFY ALL BEFORE DEPLOYING)

Plain Text

- ☐ Design Studio: Upload an image → it auto-fits within print area
- ☐ Design Studio: Drag the uploaded image → it moves smoothly
- ☐ Design Studio: Resize from corners → maintains aspect ratio
- ☐ Design Studio: Click outside print area → design deselects
- ☐ Design Studio: Change product color → container size stays EXACTLY the same
- ☐ Design Studio: Switch between views (front/back) → print area updates correct
- ☐ Design Studio: Works on iPhone Safari, Android Chrome, Desktop Chrome/Firefox
- ☐ Design Studio: Print area borders match actual product dimensions
- ☐ Design Studio: Uploaded design looks printed ON the product (blend mode)
- ☐ Frontend: All pages scroll smoothly on desktop with mouse wheel
- ☐ Frontend: All pages scroll smoothly on mobile with touch
- ☐ Frontend: No horizontal overflow on any page at any screen size
- ☐ Frontend: Product grids are responsive (2-3-4 columns based on screen)
- ☐ Frontend: All images load properly and are optimized

- ❑ Admin: Database status shows "Connected" consistently
- ❑ Admin: All admin pages are responsive on mobile
- ❑ Admin: All CRUD operations work (create/read/update/delete products)
- ❑ Performance: No console errors
- ❑ Performance: Page loads in under 3 seconds
- ❑ Performance: No layout shifts (CLS < 0.1)

EXECUTION ORDER (Follow this exactly):

1. **FIRST** — Fix the global CSS (Section 2.1 & 2.2) — this fixes scroll and overflow everywhere
2. **SECOND** — Rebuild the Design Studio interaction system (Section 1.1)
3. **THIRD** — Fix print area coordinates and auto-fit (Section 1.2 & 1.3)
4. **FOURTH** — Fix the color change sizing bug (Section 1.4)
5. **FIFTH** — Add design blend mode for realism (Section 1.5)
6. **SIXTH** — Fix page-by-page responsiveness (Section 2.3 & 2.4)
7. **SEVENTH** — Fix admin panel database and responsiveness (Section 3)
8. **EIGHTH** — Apply premium UI polish (Section 4)
9. **NINTH** — Run through the testing checklist (Section 5)

REMEMBER: Do NOT stop halfway. Do NOT say "I've made some improvements." Complete ALL sections. Every single fix listed above must be implemented. The site must work flawlessly on mobile, tablet, and desktop. The design studio must allow full touch/mouse interaction with uploaded designs. Print areas must be accurate. No more layout shifts. No more scroll issues. Premium quality. Zero errors.